ozon tech
образование

ИИ и Kubernetes





Иван Вострилов

Ozon Tech

Старший разработчик
«Compute Платформы»

План лекции

01 Зачем нужен Kubernetes

02 Разворачиваем приложение — Pod

03 Даём постоянный сетевой адрес — Service

04 Поддерживаем стабильный uptime — Deployment

05 Обновляем приложение без downtime

06 ИИ-агент в K8S

БЛОК 01

Что такое Kubernetes и зачем он нашему приложению

Приложение работает на ноутбуке



localhost:8080

Калькулятор доставки

Куда

Зион

Сумма заказа

1 200 морковок

Рассчитать

Доставка

100 морковок

Покупатели оформляют заказы круглосуточно



Работать 24/7

Даже когда ноутбук разработчика выключен.

Выдерживать рост нагрузки

В распродажу заказов становится больше.

Обновляться без остановки

Новые возможности появляются,
пока покупатели оформляют заказы.

Приложению нужен постоянный дом



Серверы есть. Как управлять запуском приложения?

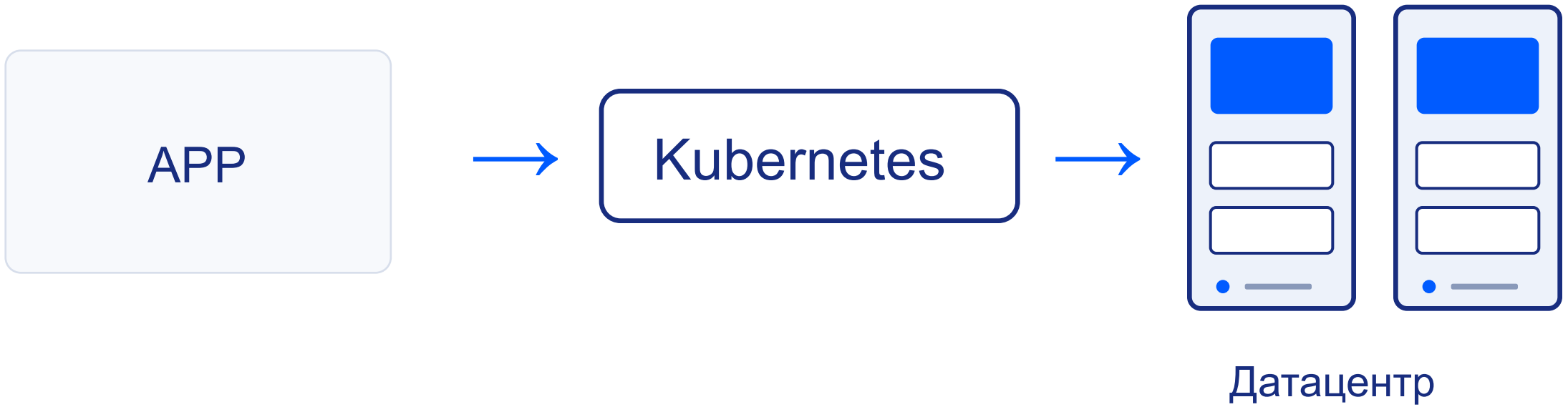


На каком из серверов запустить приложение?

Как восстановить работу, если сервер выйдет из строя?

Как обновить приложение, сохранив доступность?

Поручаем управление Kubernetes



Kubernetes, или k8s

Google · 2014

Проект опубликован как open source

Опыт Google

Идеи внутренней системы Borg

Go

Язык реализации

Используется по всему миру

Для рабочих приложений и сервисов

k ubernete s → k8s

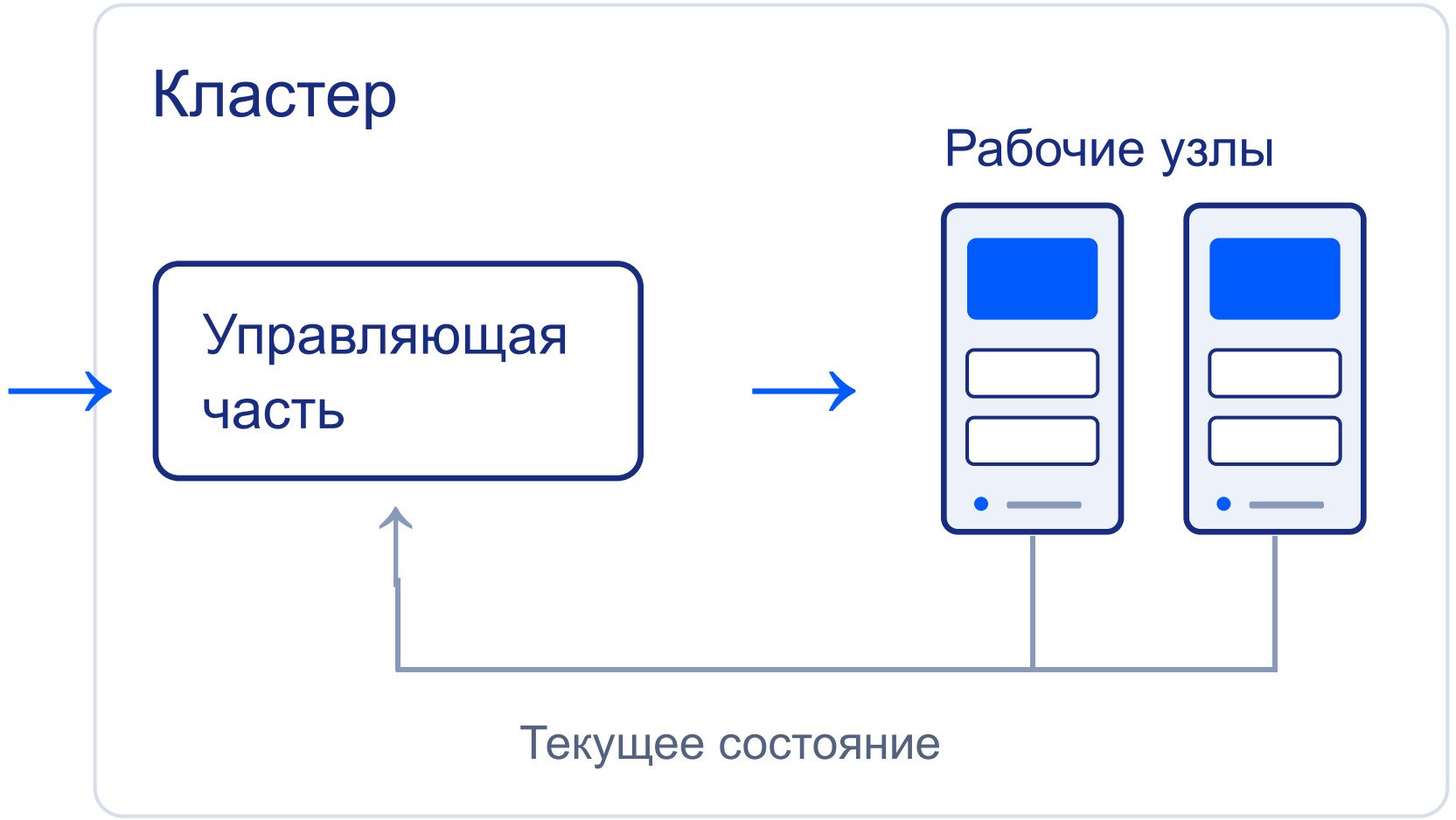
8 букв

Как устроен кластер Kubernetes

Желаемое состояние

Калькулятор доставки

Код
Зависимости
Спецификация



Зачем нужен Kubernetes?

- Размещает приложения на серверах с учётом ресурсов.
- Даёт приложениям постоянные сетевые адреса внутри кластера.
- Поддерживает заданное число экземпляров приложения.
- Заменяет экземпляры, потерянные при сбоях.
- Постепенно обновляет приложение до новой версии.

План лекции

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod

03 Даём постоянный адрес — Service

04 Поддерживаем число экземпляров — Deployment

05 Обновляем приложение

06 ИИ-агент в K8S

БЛОК 02

Запускаем приложение на сервере под управлением K8S

Как доставить приложение на сервер (ноду)?

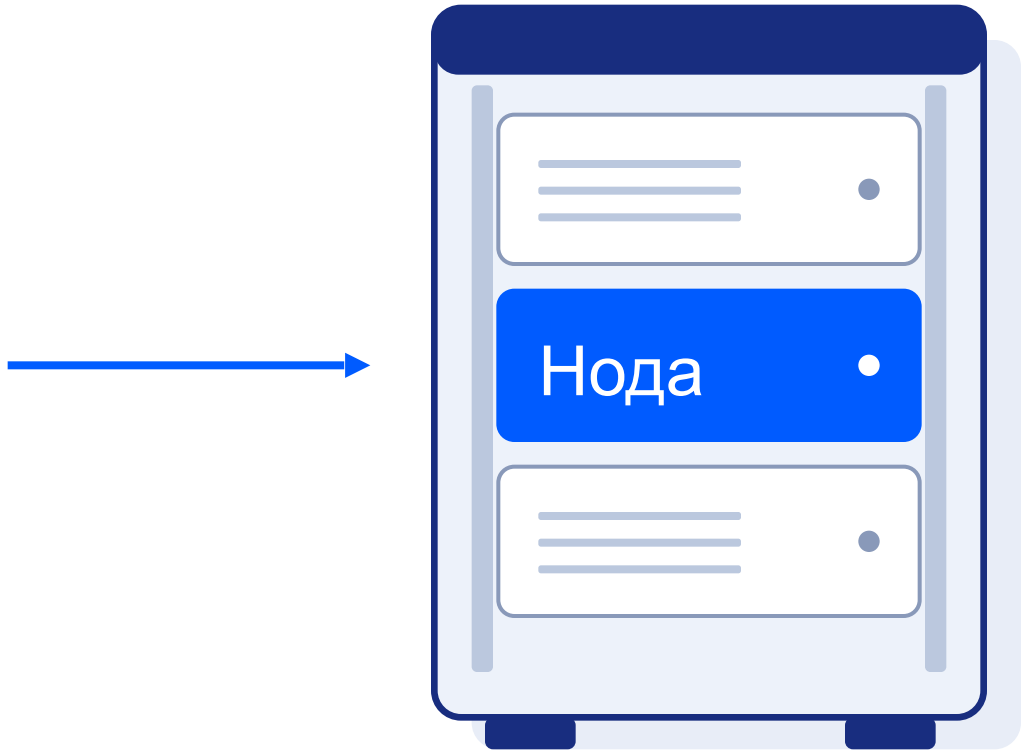
Docker-образ

app

Код приложения

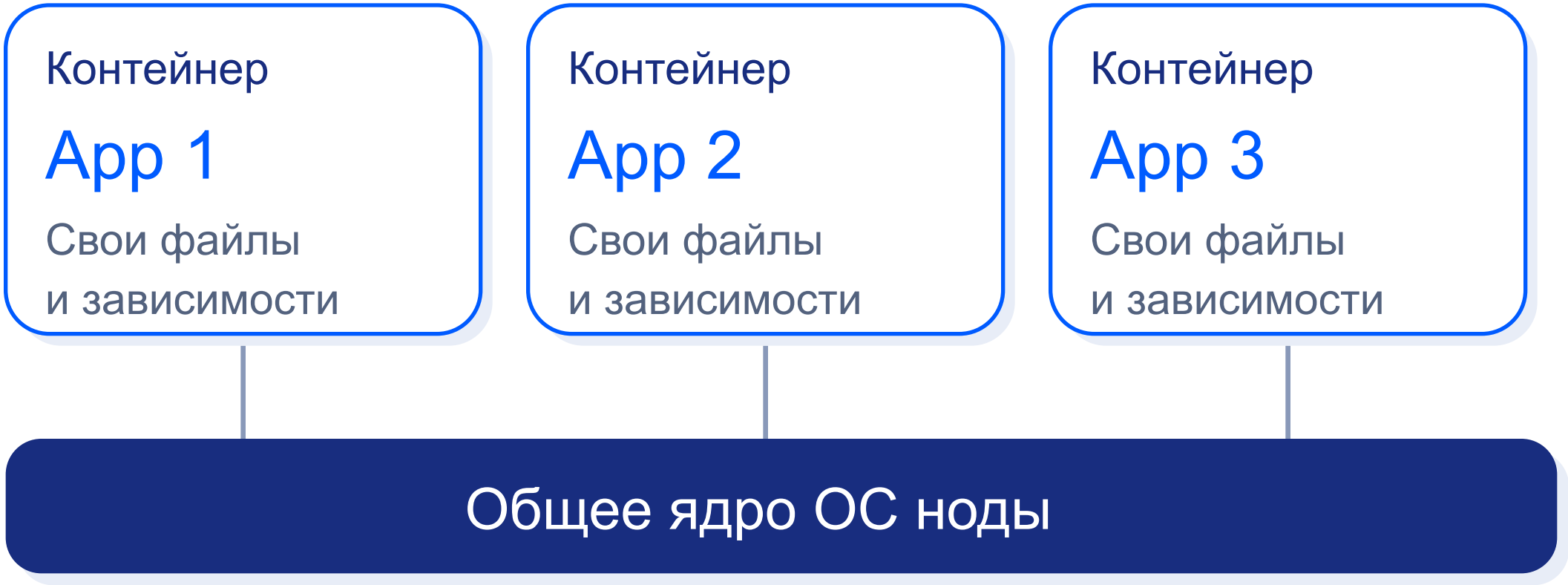
Зависимости

Ресурсы



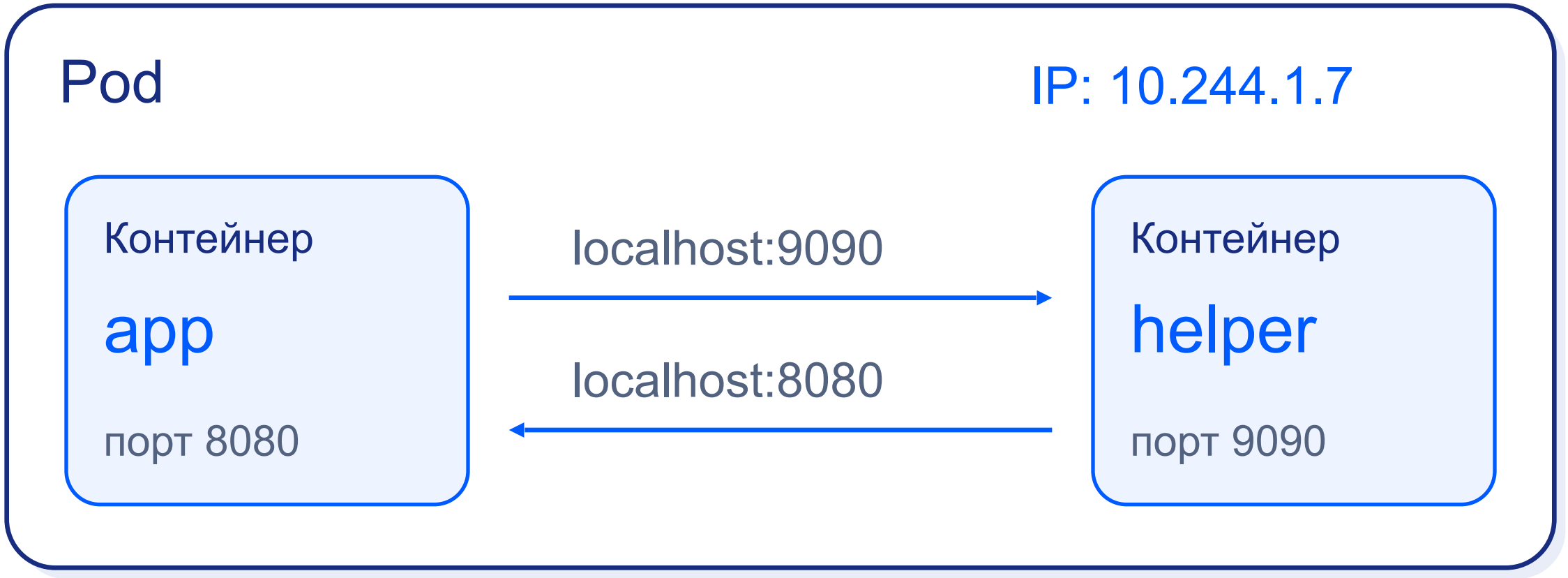
Серверная стойка

Как запускать разные приложения на одном сервере?

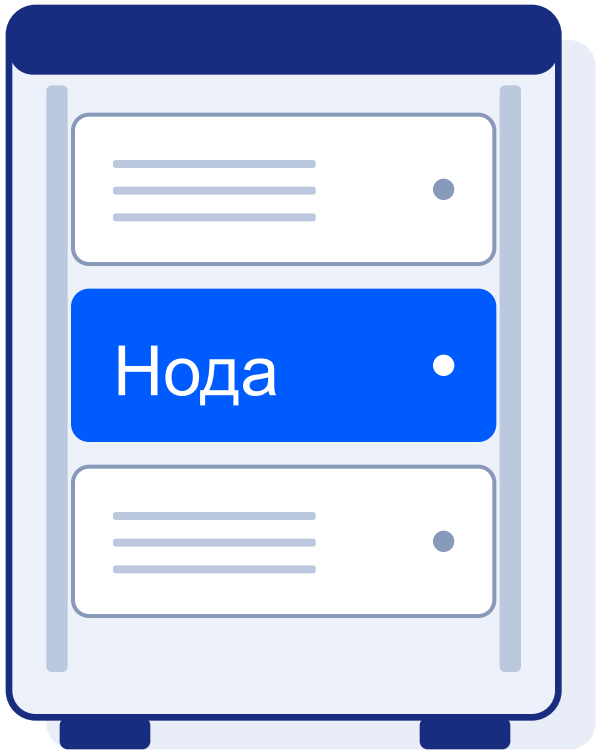


Изоляция защищает окружение приложений

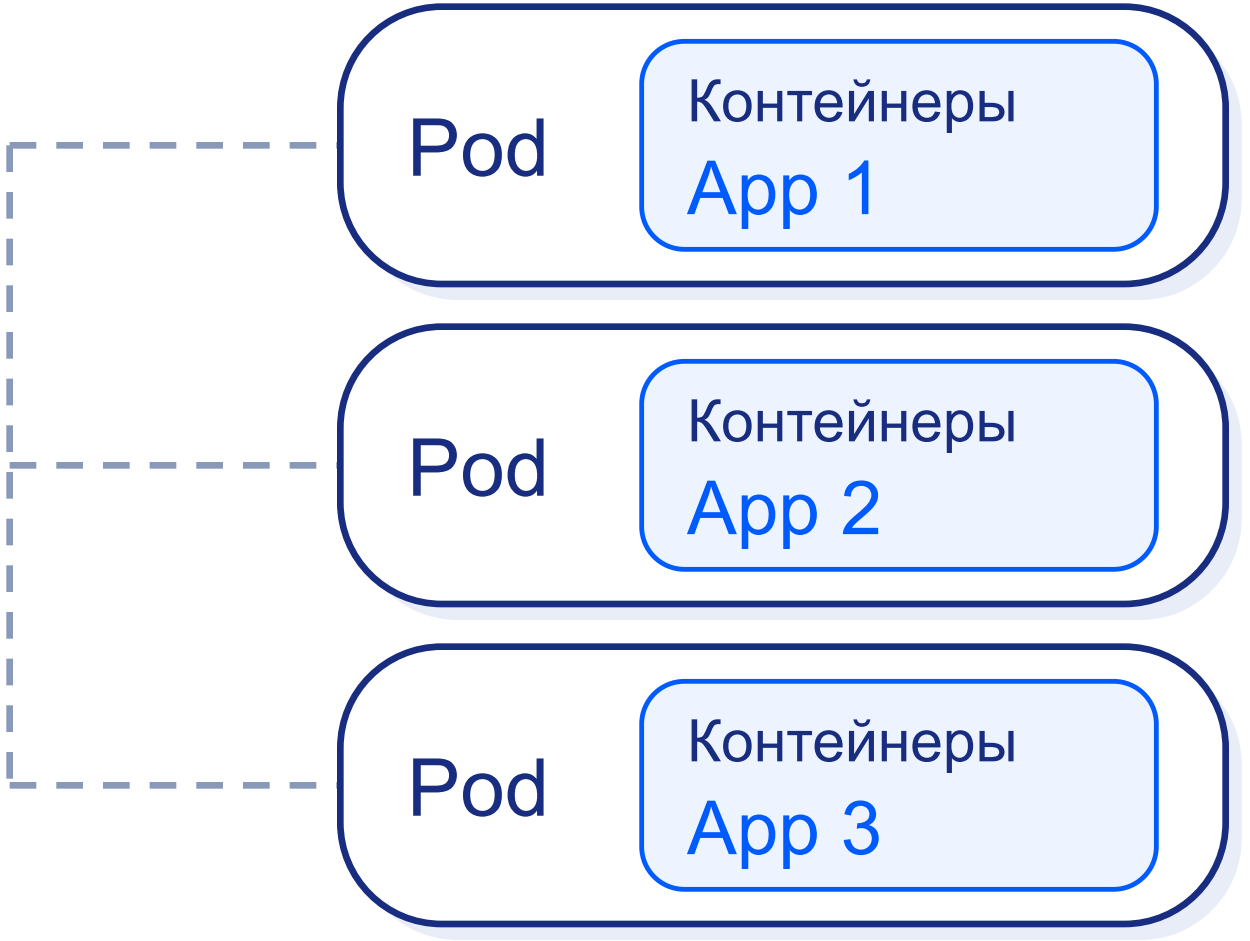
Pod - базовая единица развёртывания



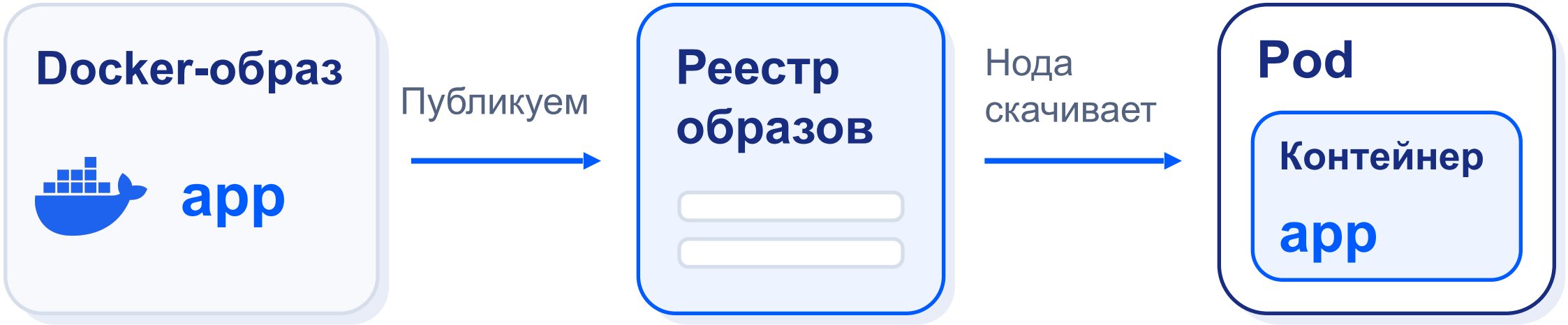
На одной ноде могут работать несколько Pods



Серверная стойка

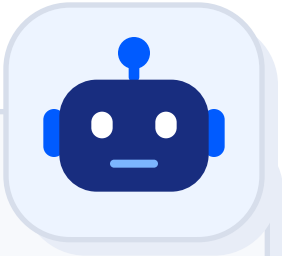


Что нужно знать, чтобы запустить Pod?



Что запускаем	registry.marketplace.example/delivery-calc:1.0
Как подключиться	Порт 8080
Где запускаем	ctx marketplace-dev / namespace delivery
Ресурсы	CPU и память: requests / limits

Ставим задачу агенту



Подготовь Pod для калькулятора доставки.
Один контейнер из образа:
registry.marketplace.example/delivery-calc:1.0

Приложение слушает порт 8080.
Контекст: marketplace-dev. Namespace: delivery.

requests — 1 CPU, 1Gi памяти; limits — 2 CPU, 2Gi памяти.

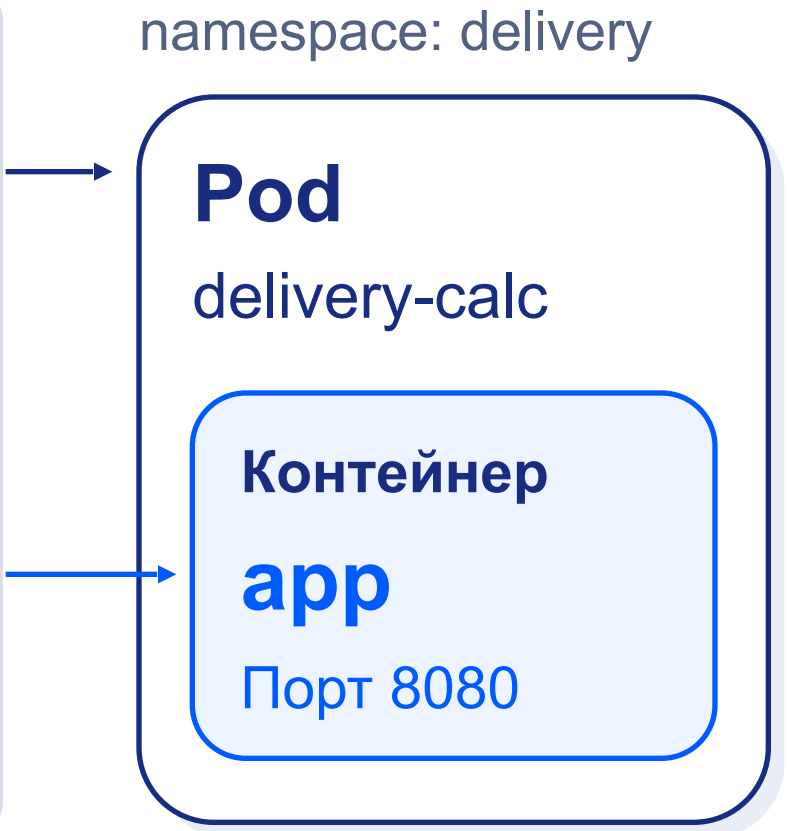
Сохрани манифест в pod.yaml и объясни поля.

Пока не применяй манифест.

Как описать наш Pod?

pod.yaml · фрагмент

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: delivery-calc
  namespace: delivery
spec:
  containers:
  - name: app
    image: …/delivery-calc:1.0
    ports:
    - containerPort: 8080
```



Сколько ресурсов дать контейнеру?

pod.yaml · фрагмент контейнера app

```yaml
resources:
  requests:
    cpu: "1"
    memory: "1Gi"
  limits:
    cpu: "2"
    memory: "2Gi"
```

requests

1 CPU · 1Gi памяти

Учитываются при выборе ноды

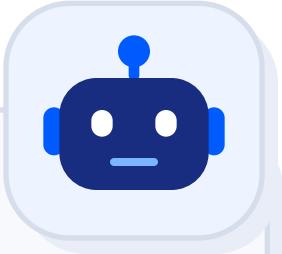
limits

2 CPU · 2Gi памяти

Ограничивают потребление

Контейнер может использовать больше requests — в пределах limits.

Просим агента запустить Pod



Примени pod.yaml в контексте marketplace-dev, namespace delivery.

Покажи команду и наблюдай за запуском Pod.

Агент выполняет

```
kubectl --context marketplace-dev -n delivery \
  apply -f pod.yaml

pod/delivery-calc created
```

Пример вывода · объект Pod создан, запуск продолжается

Наблюдаем за запуском

Агент запускает наблюдение · -w показывает изменения

```
kubectl --context marketplace-dev -n delivery \
  get pods -w

NAME             READY   STATUS              RESTARTS   AGE
delivery-calc    0/1     Pending             0          0s
delivery-calc    0/1     ContainerCreating   0          1s
delivery-calc    1/1     Running             0          4s
```

Pending

Выбирается нода,
готовится контейнер

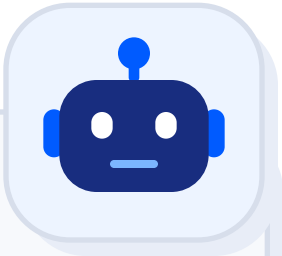


Running

Контейнер app запущен
на выбранной ноде

Пример вывода · ответ приложения проверим следующим шагом

Просим агента подготовить проверку



Проверь готовность Pod delivery-calc.
Открой доступ к приложению на localhost:8080
для проверки в браузере.

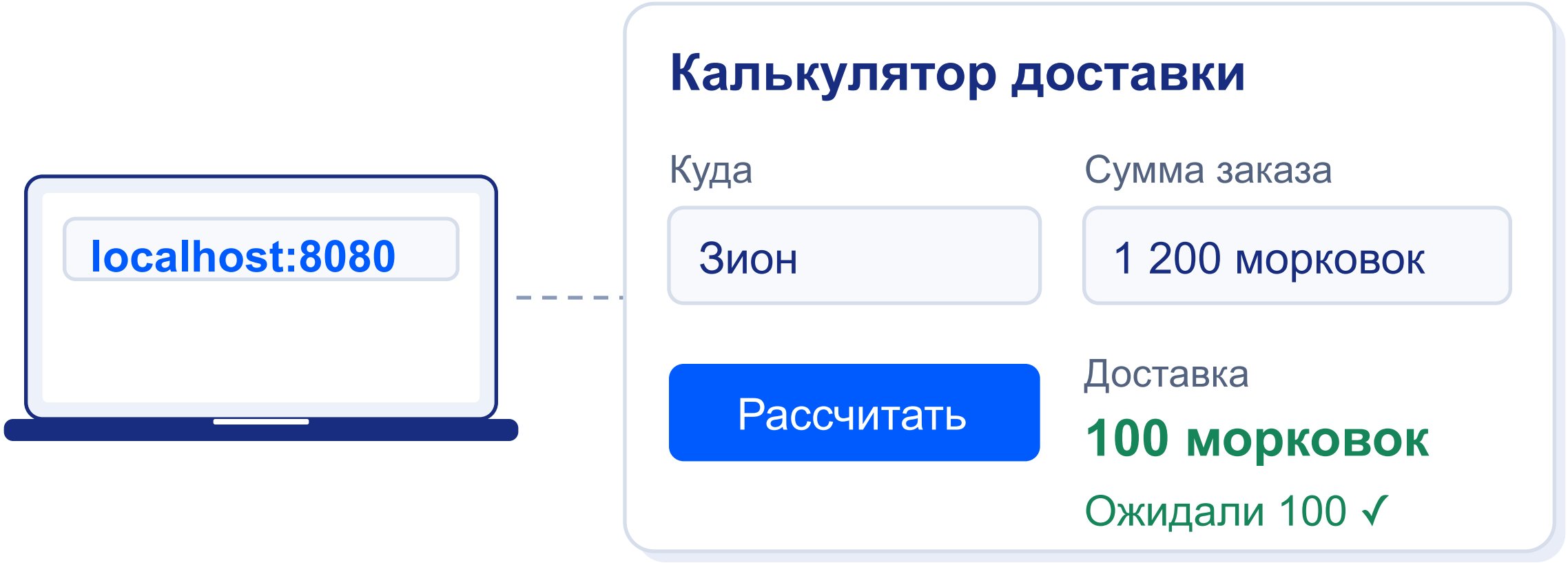
Агент выполняет · пример команд

```
kubectl --context marketplace-dev -n delivery \
  wait --for=condition=Ready pod/delivery-calc --timeout=60s

kubectl --context marketplace-dev -n delivery \
  port-forward pod/delivery-calc 8080:8080
```

Проверяем приложение

Браузер на ноутбуке, приложение — в Pod



План лекции

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod ✓

03 Даём постоянный адрес — Service

04 Поддерживаем число экземпляров — Deployment

05 Обновляем приложение

06 ИИ-агент в K8S

БЛОК 03

Выделяем постоянный сетевой адрес до сервиса

Как клиенту не потерять сервис?

Оформление заказа

Рассчитать доставку



Удалён

Pod калькулятора

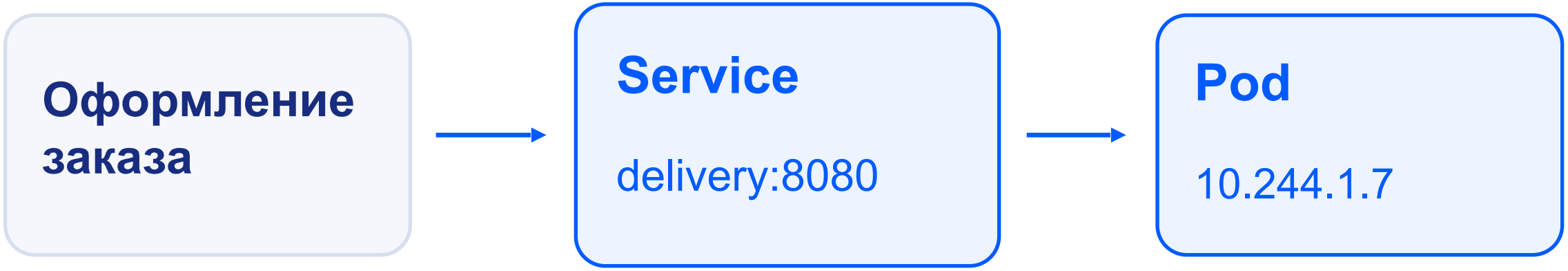
10.244.1.7:8080

Pod меняется. Адрес для клиента должен сохраняться.

Pod калькулятора

10.244.2.9:8080

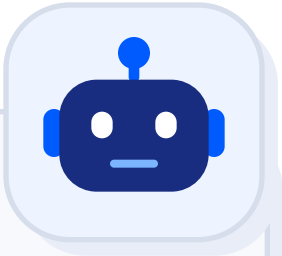
Service — постоянная точка доступа



Клиент обращается к имени Service

ClusterIP · доступ внутри кластера

Ставим задачу агенту



Подготовь постоянный адрес калькулятора: delivery:8080 — для клиента внутри кластера.

Контекст: marketplace-dev. Namespace: delivery.
Pod из pod.yaml слушает порт 8080.

Создай service.yaml с Service ClusterIP.
Добавь метки в pod.yaml, объясни selector и порты.

Пока не применяй манифесты.

Как Service находит нужный Pod?

service.yaml · фрагмент

```
spec:
  selector:
    app: delivery
```



pod.yaml · фрагмент

```
metadata:
  labels:
    app: delivery
```

Метка описывает Pod. Selector выбирает его.

Как описать Service?

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: delivery
  namespace: delivery
spec:
  type: ClusterIP
  selector:
    app: delivery
  ports:
  - port: 8080
    targetPort: 8080
```

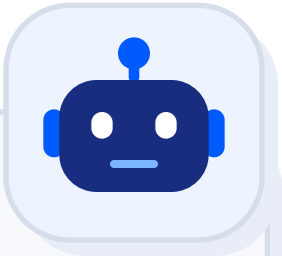
port: 8080

Порт Service

targetPort: 8080

Порт приложения в Pod

Применяем и проверяем



Примени pod.yaml и service.yaml.

Проверь расчёт через Service из тестового клиента в namespace delivery.

Агент выполняет

```
kubectl --context marketplace-dev -n delivery \
  apply -f pod.yaml -f service.yaml

service/delivery created
```

Пример вывода · затем проверяем расчёт через Service

Pod работает. Почему запрос не проходит?

Пример ошибки · несовпадение меток

Service · selector

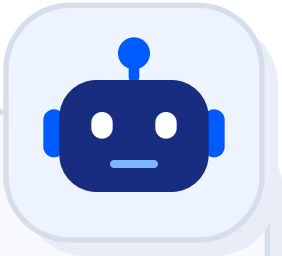
```
app: calculator
```

Pod · labels

```
app: delivery
```

Исправить selector → применить → повторить запрос

Меняем Pod. Адрес клиента остаётся



Удали Pod калькулятора и создай его заново.

Используй pod.yaml. Дождись готовности
и повтори прежний запрос через Service.

До замены

delivery:8080

→

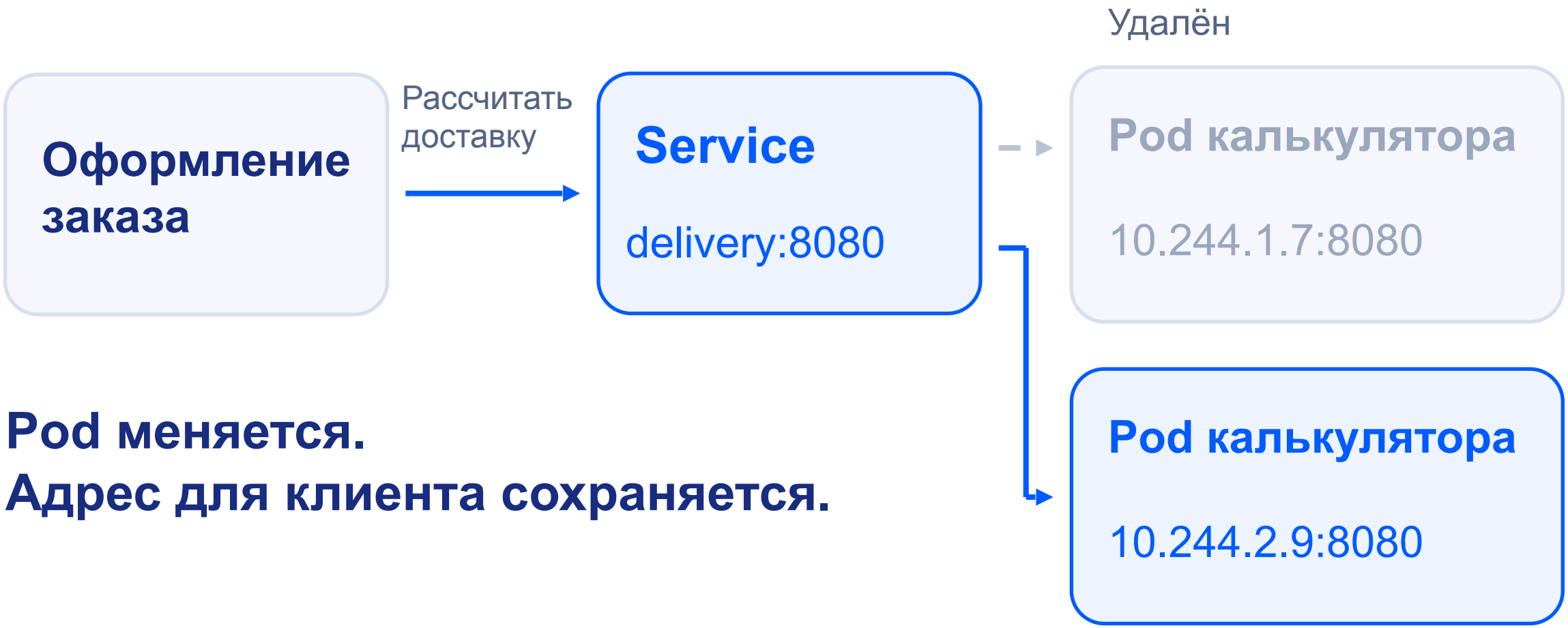
После замены

delivery:8080

Расчёт вернулся · экземпляр изменился

Пока нет готового Pod, запросы могут завершаться ошибкой

Pod новый. Адрес калькулятора прежний



Pod меняется.
Адрес для клиента сохраняется.

План лекции

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod ✓

03 Даём постоянный адрес — Service ✓

04 Поддерживаем число экземпляров — Deployment

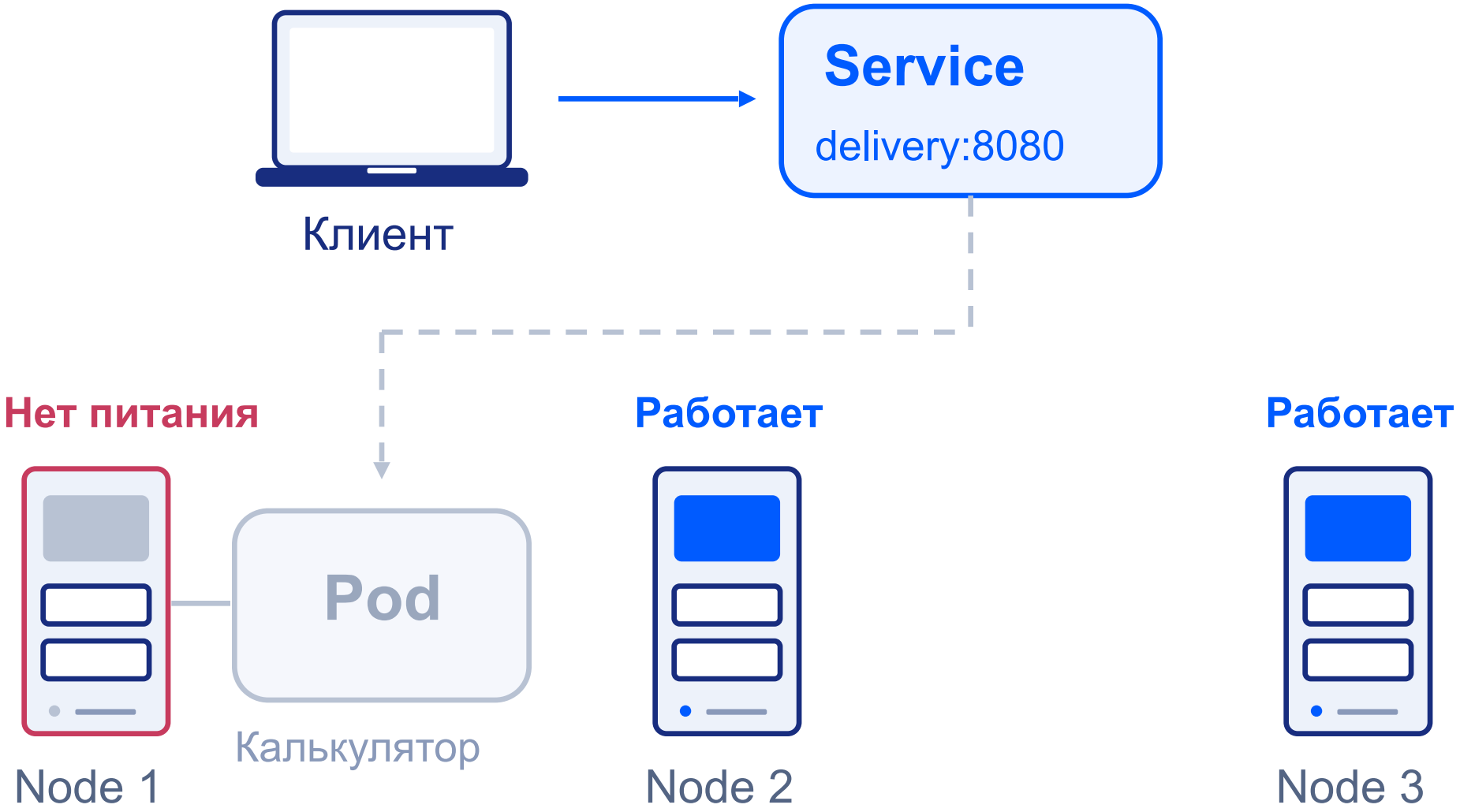
05 Обновляем приложение

06 ИИ-агент в K8S

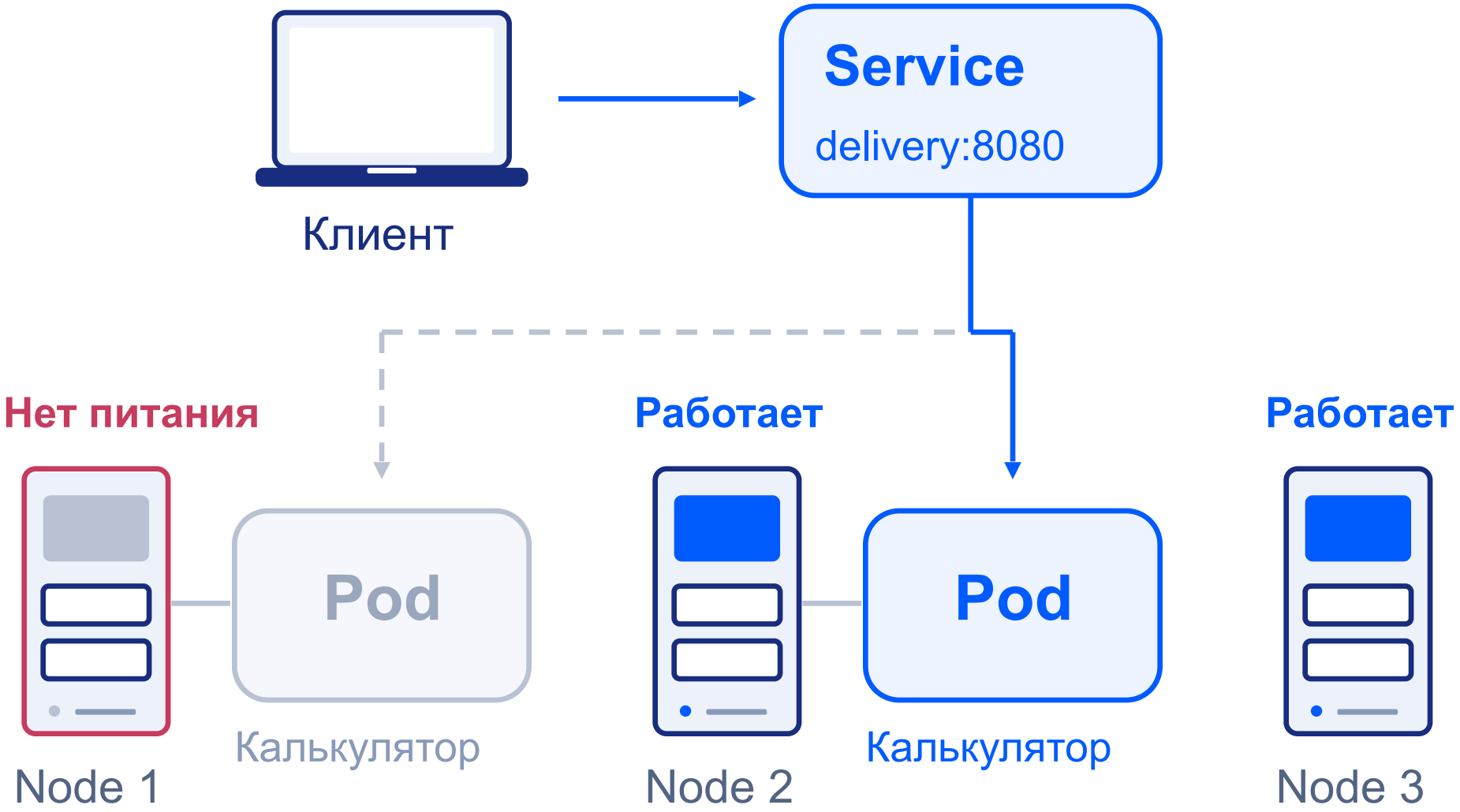
БЛОК 04

Поддерживаем доступность при северных сбоях

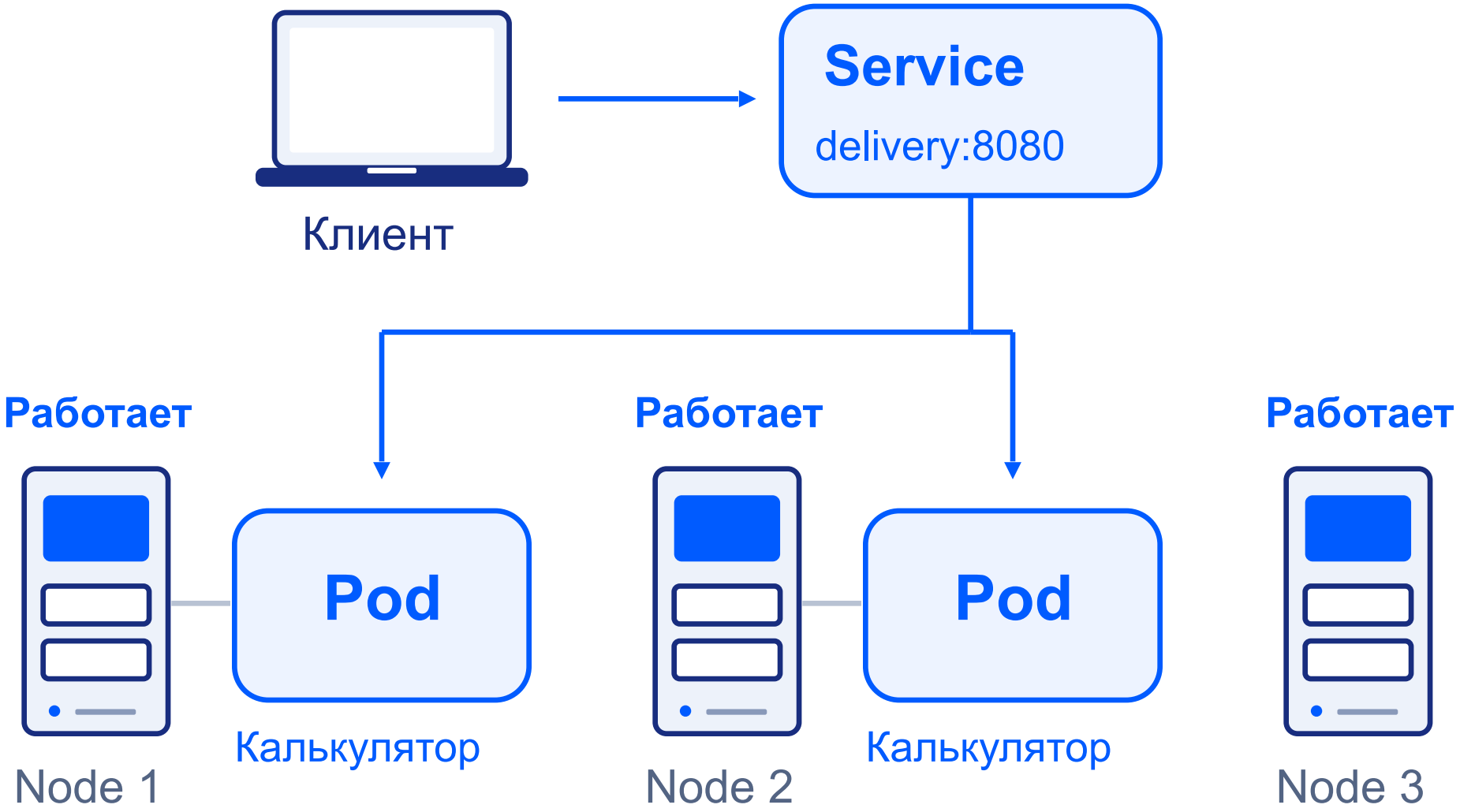
Сервер выключился. Кто ответит клиенту?



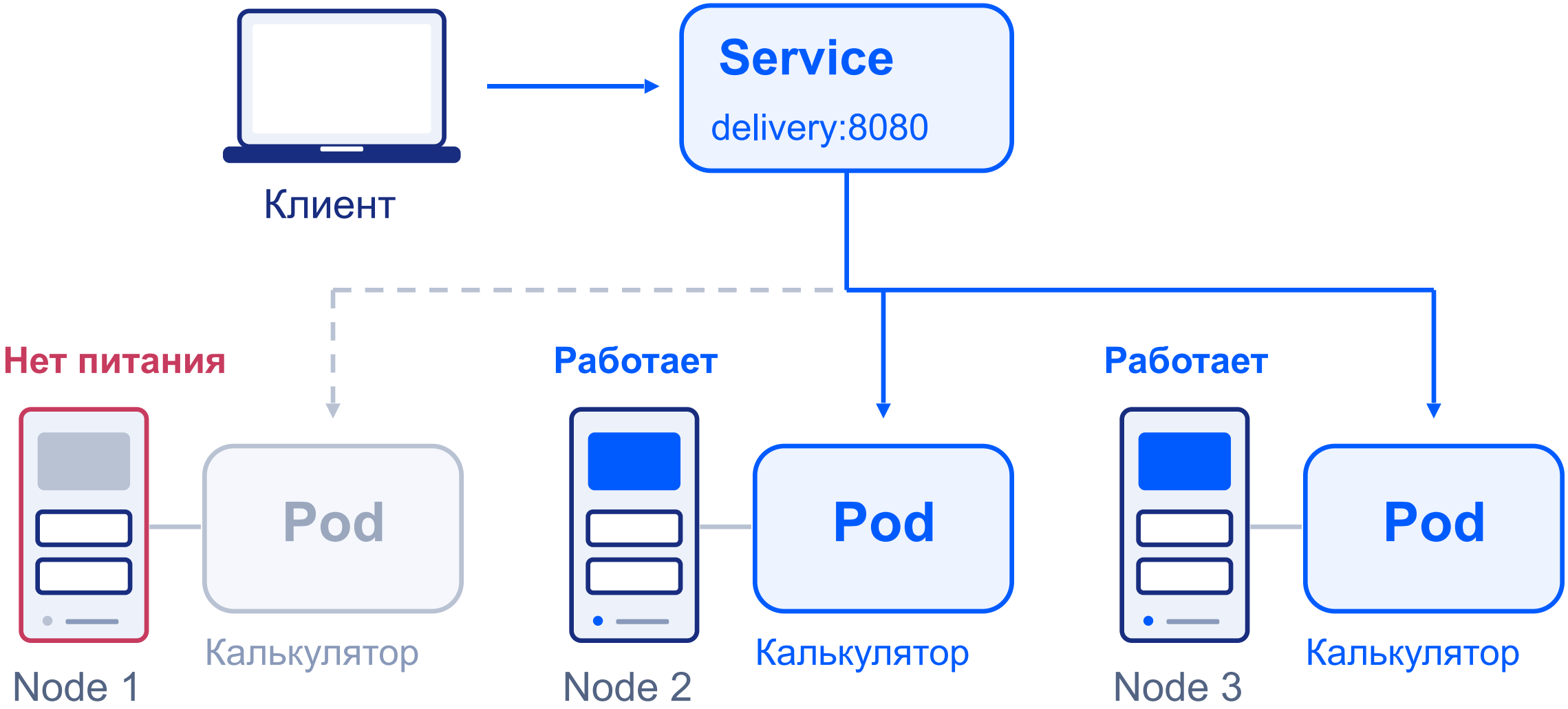
Второй экземпляр на другой ноде



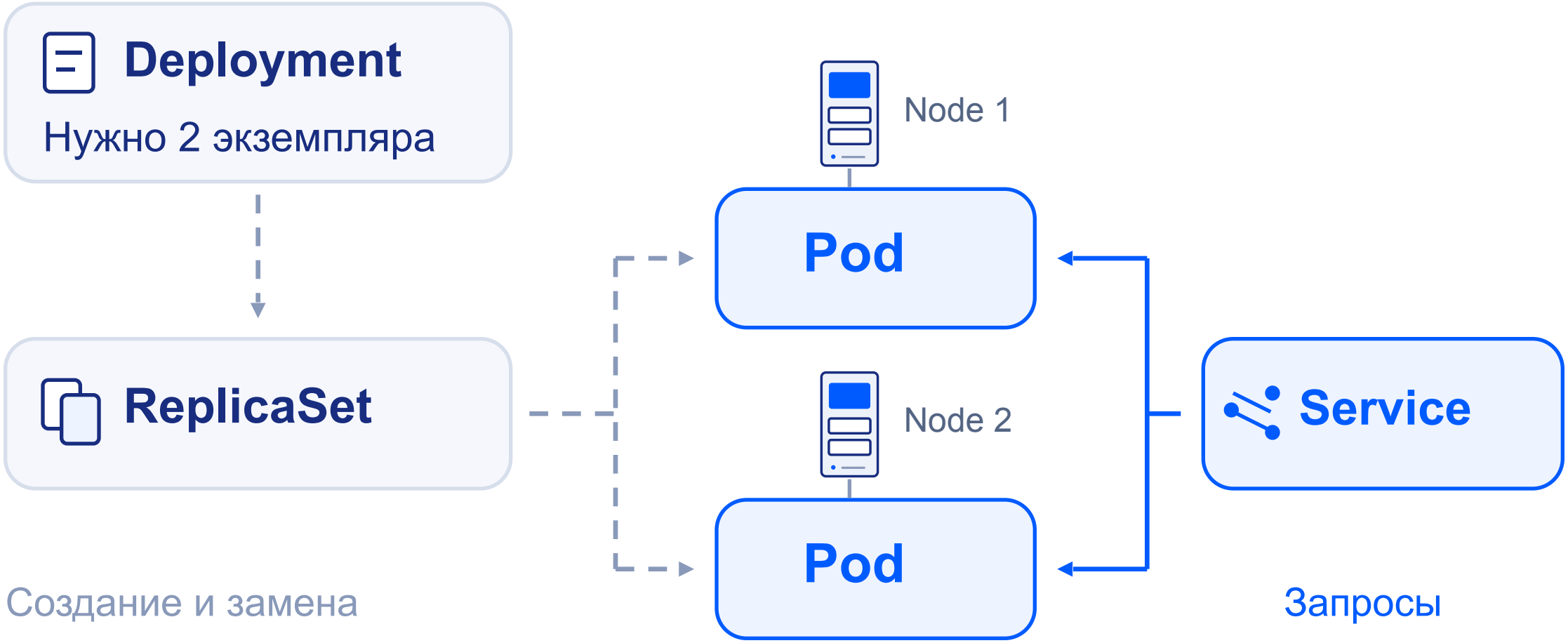
Оба экземпляра принимают запросы



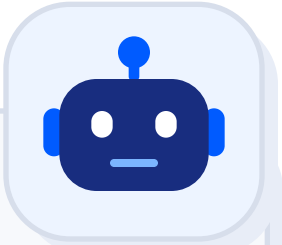
Новый экземпляр на третьей ноде



Deployment — управляем экземплярами



Просим агента повысить надёжность



Замени одиночный Pod на Deployment.

Два экземпляра калькулятора v1 на разных nodes.
Сохрани образ, порт и ресурсы из pod.yaml.
Доступ — через существующий Service delivery.

Контекст: marketplace-dev. Namespace: delivery.
Сохрани deployment.yaml и объясни изменения.
Предложи переход с одиночного Pod и проверки.

Пока не применяй манифесты.

Что изменилось в манифесте?

deployment.yaml · сокращённый фрагмент

```
kind: Deployment
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: delivery
    spec:
      containers:
      - name: app
        image: …/delivery-calc:1.0
```

replicas: 2

Нужны два экземпляра

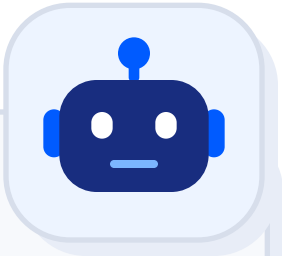
template

Знакомое описание Pod

Разные ноды

Отдельное правило размещения

Применяем и проверяем



Примени решение по согласованному плану.

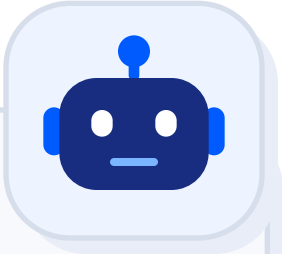
Проверь два готовых Pod на разных nodes
и повтори расчёт через Service delivery.

Агент выполняет · пример команд

```
kubectl --context marketplace-dev -n delivery \
  apply -f deployment.yaml
kubectl --context marketplace-dev -n delivery \
  rollout status deployment/delivery-calc \
  --timeout=120s
```

Проверяем: 2 Ready · разные NODE · расчёт верный

Удалим один Pod. Что произойдёт?



Удали один Pod Deployment delivery-calc.

Наблюдай за заменой. Проверь готовность
двух экземпляров и расчёт через Service.

Агент выполняет · пример команды

```
kubectl --context marketplace-dev -n delivery \
  delete pod <имя-Pod>
```

Удалённый Pod → новый Pod → снова 2 Ready

Удалили Pod — появилась замена

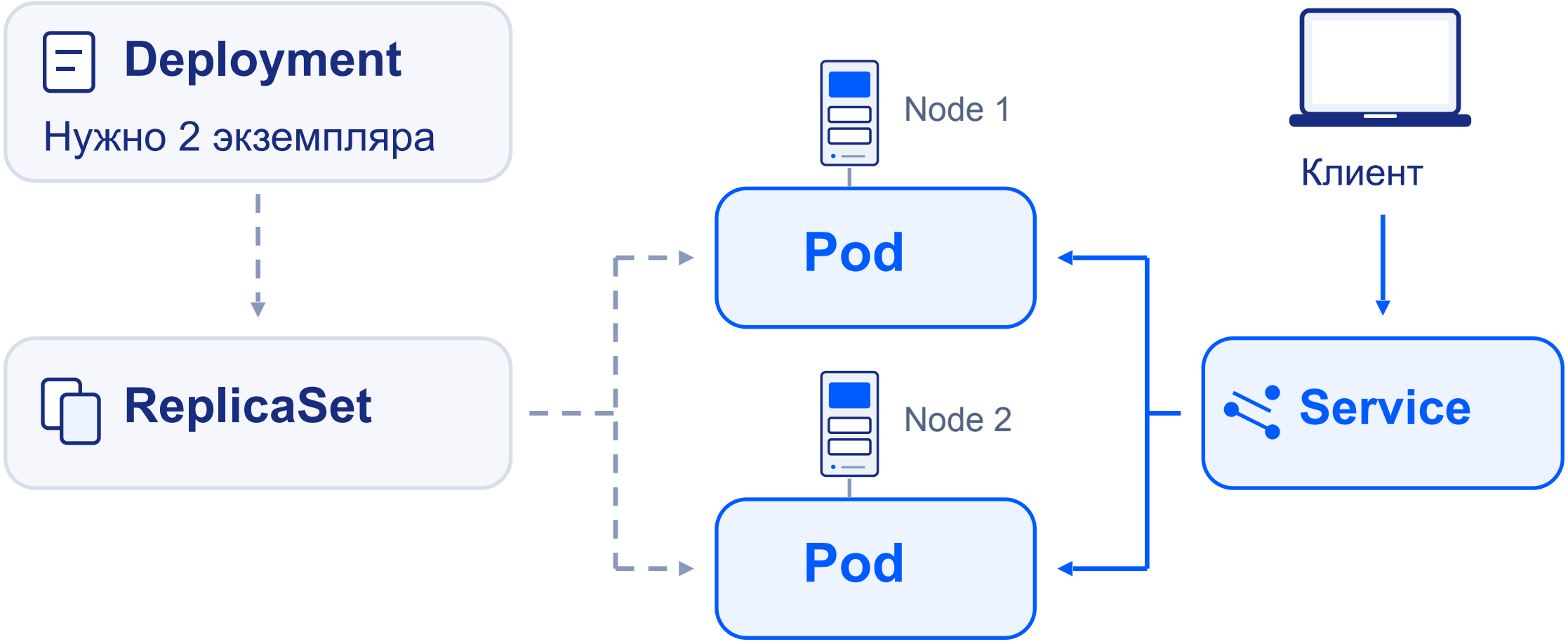
Агент наблюдает · пример команд и вывода

```
kubectl --context marketplace-dev -n delivery \
  get pods -l app=delivery -w

NAME                              READY   STATUS
delivery-calc-abc12-k7m2p         1/1     Running
delivery-calc-abc12-r8v4n         1/1     Running
delivery-calc-abc12-k7m2p         1/1     Terminating
delivery-calc-abc12-t6x9q         0/1     Pending
delivery-calc-abc12-t6x9q         0/1     ContainerCreating
delivery-calc-abc12-t6x9q         1/1     Running
```

Другой Pod · снова два готовых экземпляра

Два экземпляра. Замена — автоматически



Задаём число экземпляров — Kubernetes поддерживает его

План лекции

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod ✓

03 Даём постоянный адрес — Service ✓

04 Поддерживаем число экземпляров — Deployment ✓

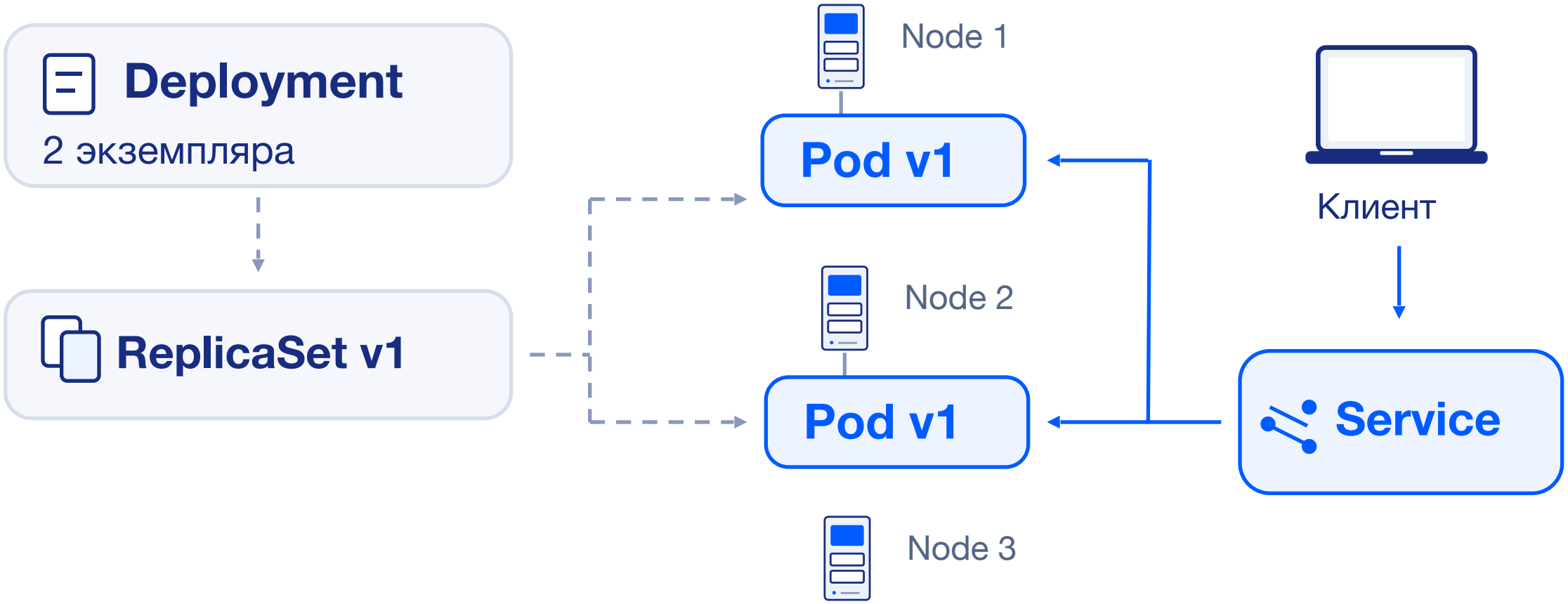
05 Обновляем приложение

06 ИИ-агент в K8S

БЛОК 05

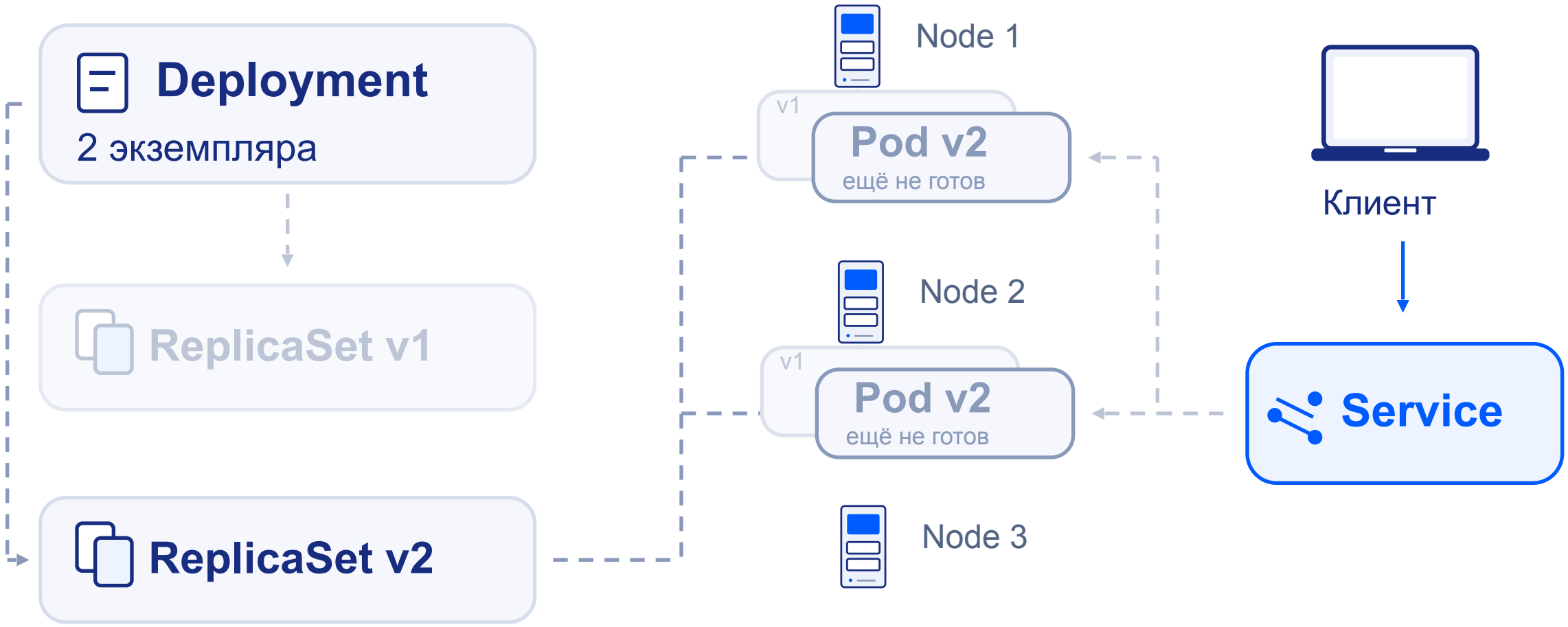
Поддерживаем доступность при обновлении

Обновлять без downtime



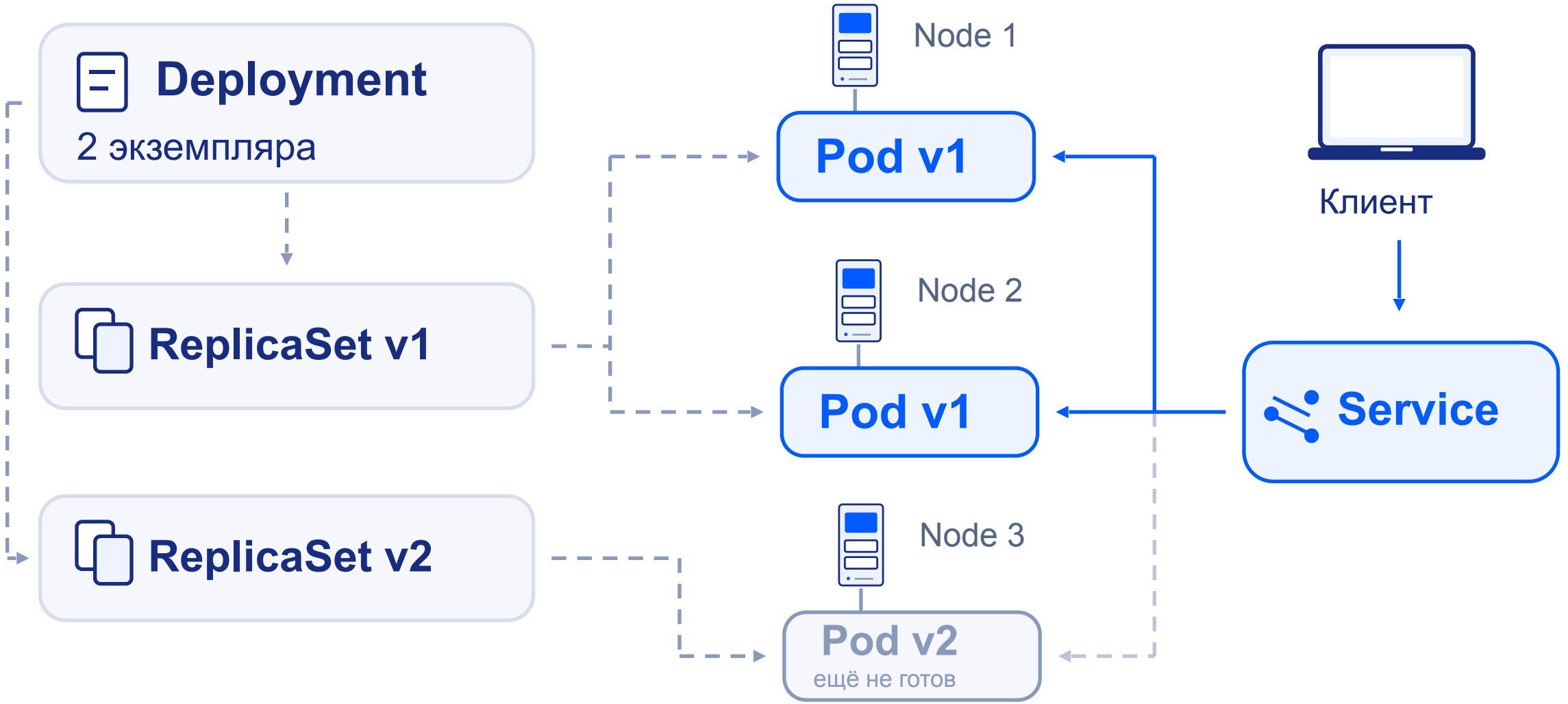
v1 → v2: добавляем экспресс-доставку

Удалили оба — отвечать некому



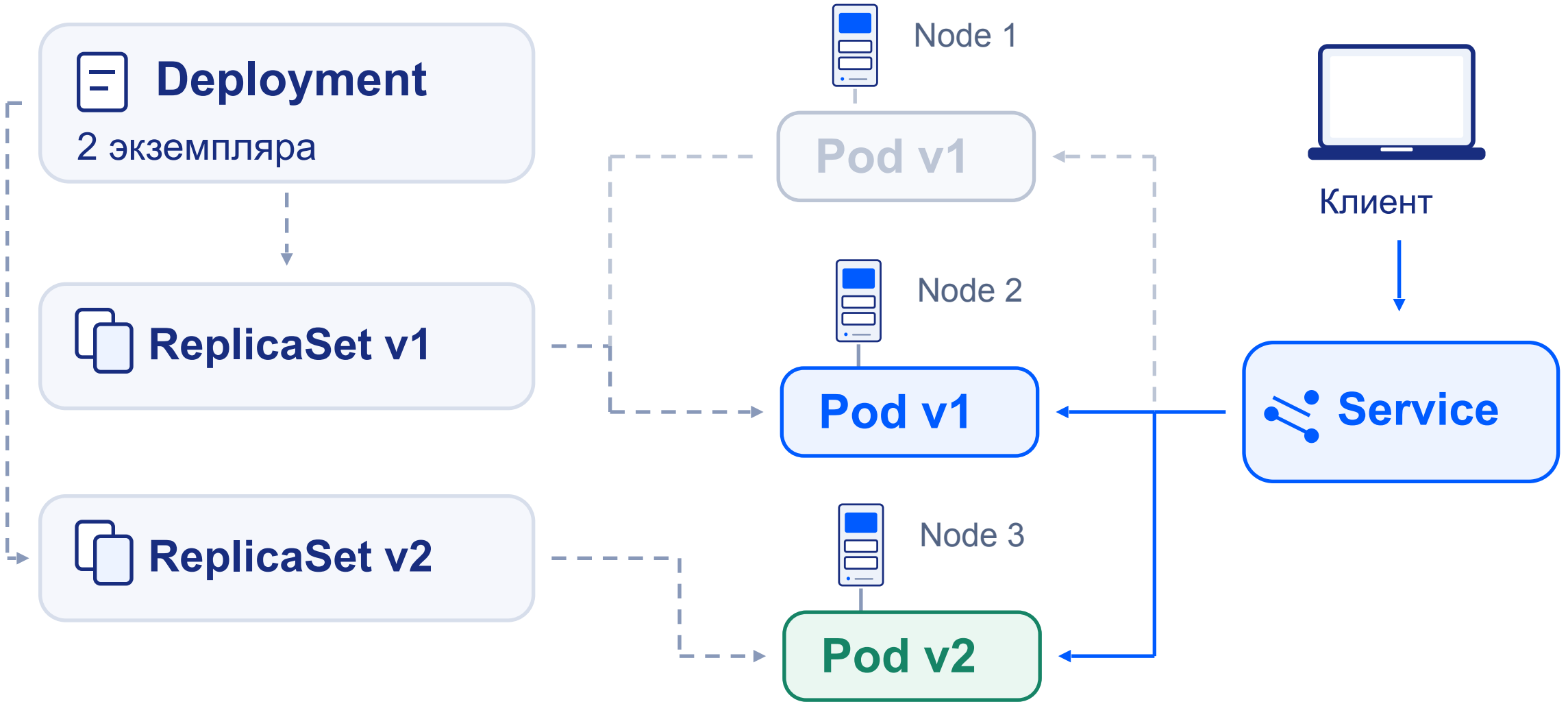
v1 удалены · v2 запускаются · downtime

Сначала запускаем замену



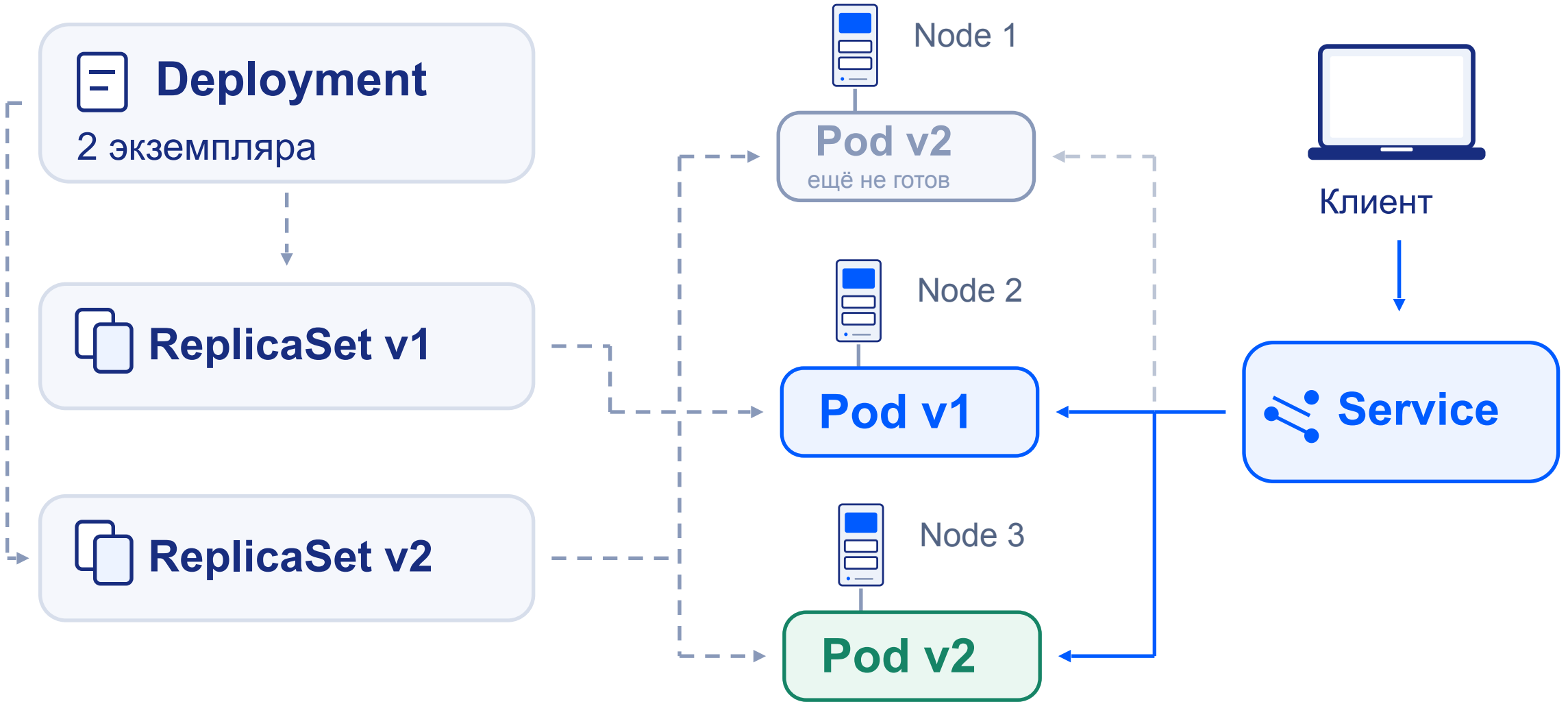
Два v1 отвечают, v2 готовится

Готов новый — убираем старый



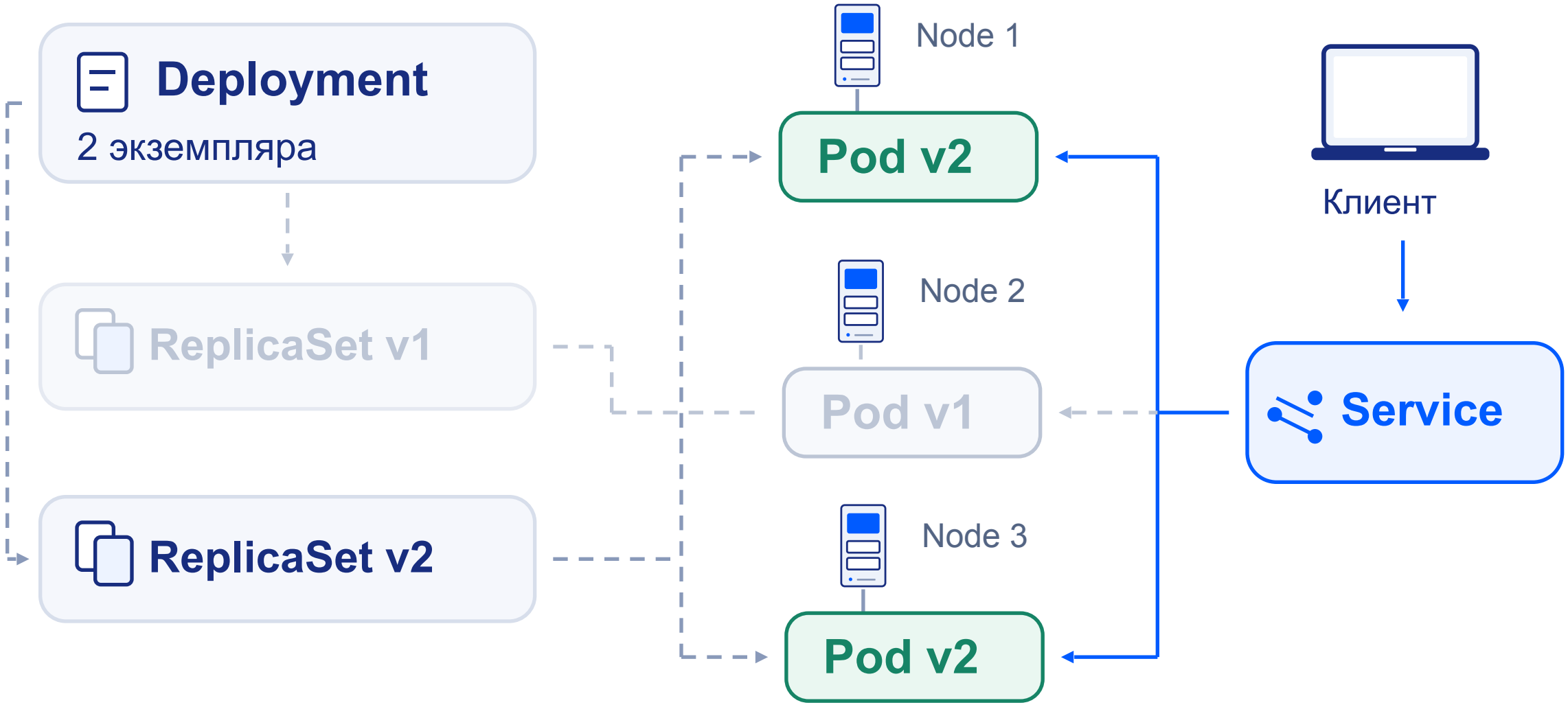
Service подключил v2 · один v1 завершён

Повторяем тот же шаг



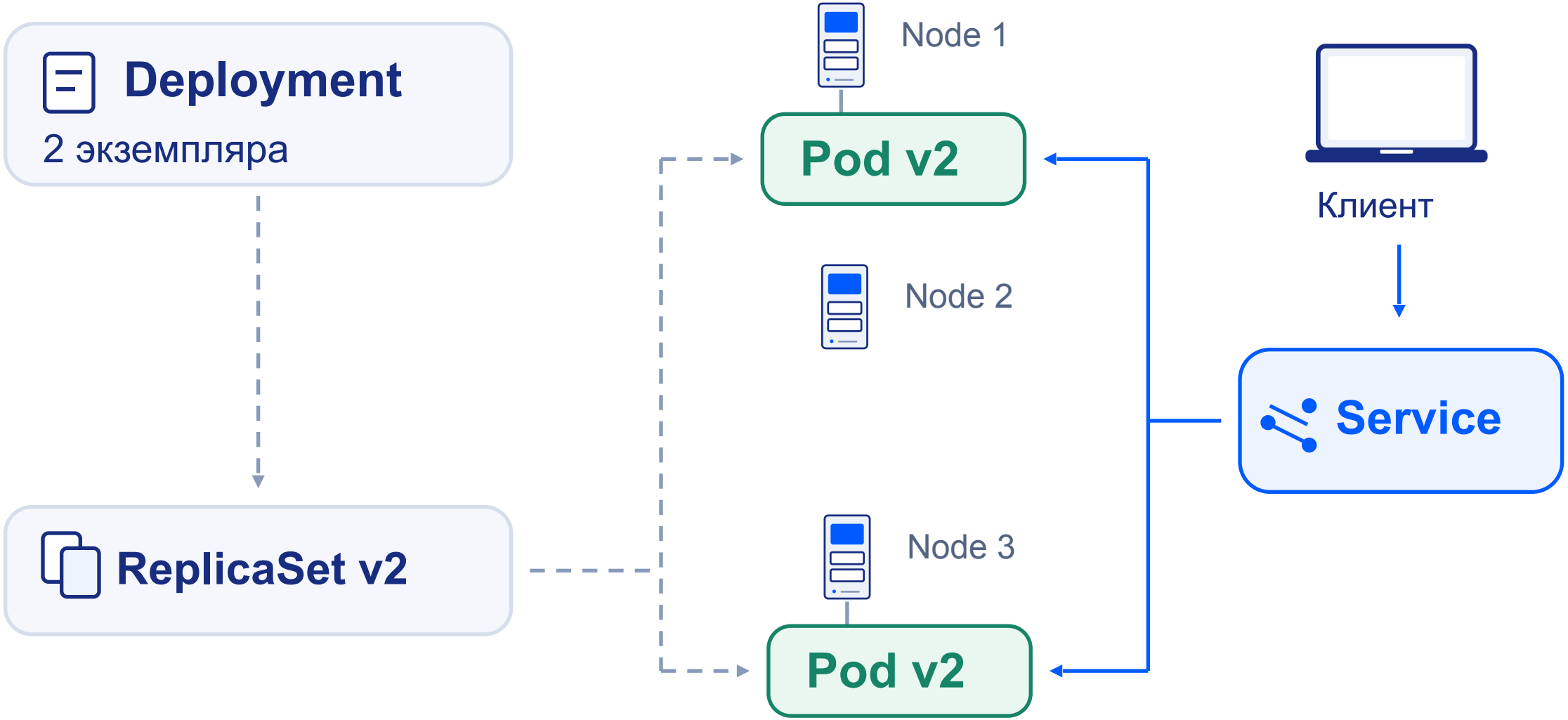
Запускаем ещё один v2 → ждём готовности

Теперь оба экземпляра — v2



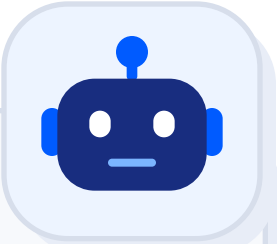
Старый ReplicaSet: 0 Pods · новый: 2

Обновление завершено



Тот же Service · два готовых Pod v2

Задаём агенту правила обновления



Настрой правила обновления Deployment.

Два доступных экземпляра во время обновления.

Не более одного дополнительного Pod.

Сохрани текущий образ v1 и остальные настройки.

Проверь и настрой readinessProbe.

Покажи diff deployment.yaml и план проверок.

Пока не применяй манифесты.

Задаём правила обновления

```yaml
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
```

0 недоступных

Сохраняем 2 доступных Pod

+1 дополнительный

Место для новой версии

Фрагмент YAML · правила будущего обновления

Запущен ≠ готов принимать запросы

```
spec:
  template:
    spec:
      containers:
      - name: app
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          periodSeconds: 2
```

Проверка успешна

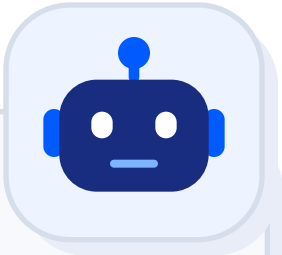
Service направляет запросы

/ready — пример контракта

Endpoint должен проверять готовность приложения

Теперь обновляем приложение на v2

Образ v2 уже загружен в registry



Обнови deployment/delivery-calc на образ:

`registry.marketplace.example/delivery-calc:2.0`

Примени манифест с подготовленными правилами.

Проверь расчёт при обновлении и экспресс в конце.

Агент выполняет · пример команд

```
kubectl --context marketplace-dev -n delivery \
  apply -f deployment.yaml
kubectl --context marketplace-dev -n delivery \
  rollout status deployment/delivery-calc --timeout=180s
```

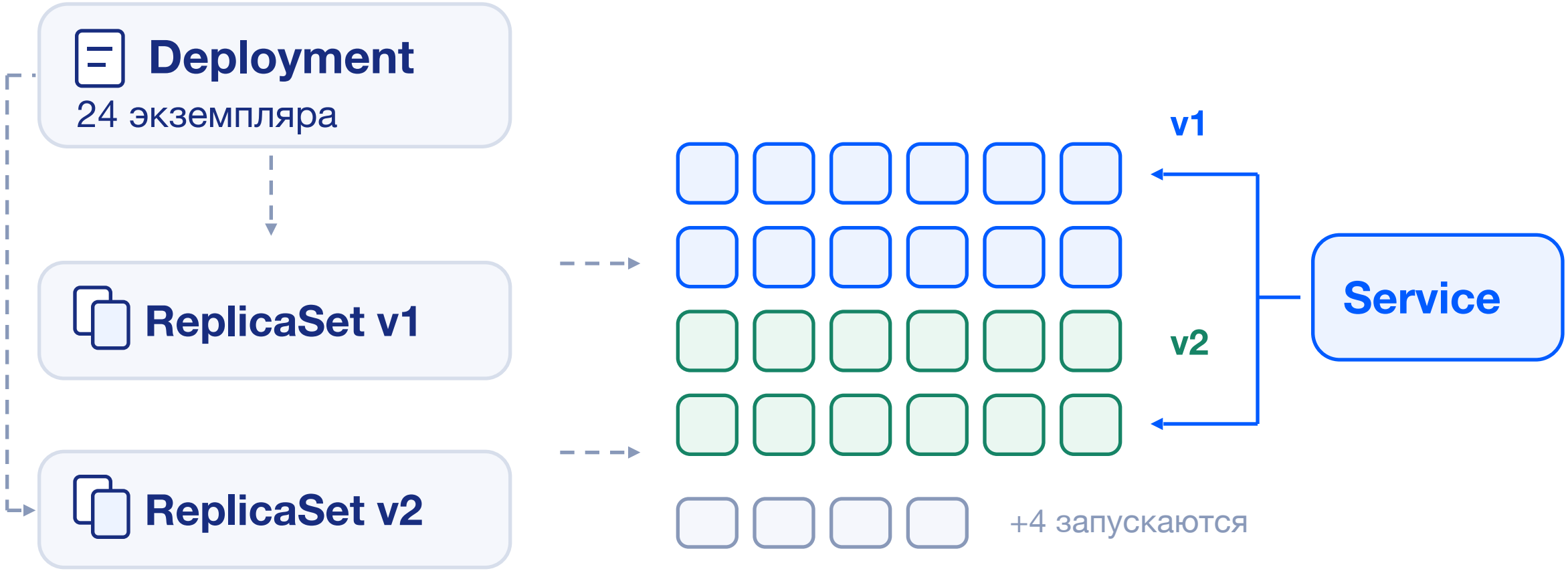
Наблюдаем постепенную замену

Пример вывода · watch запущен до обновления

```
kubectl --context marketplace-dev -n delivery \
  get pods -l app=delivery -w
NAME                              READY   STATUS
delivery-calc-abc12-k7m2p          1/1     Running
delivery-calc-abc12-r8v4n          1/1     Running
delivery-calc-def34-t6x9q          0/1     Running
delivery-calc-def34-t6x9q          1/1     Running
delivery-calc-abc12-k7m2p          1/1     Terminating
delivery-calc-def34-h4p8s          1/1     Running
delivery-calc-abc12-r8v4n          1/1     Terminating
```

Клиентские запросы проверяем отдельно

Тот же механизм — десятки Pods



24 реплики · maxSurge: 4 · maxUnavailable: 0

План лекции

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod ✓

03 Даём постоянный адрес — Service ✓

04 Поддерживаем число экземпляров — Deployment ✓

05 Обновляем приложение ✓

06 ИИ-агент в K8S

Исходники помогают агенту понимать Kubernetes

- Проверяет логику по исходному коду.
- Сверяется с документацией.
- Использует это при подготовке манифестов.

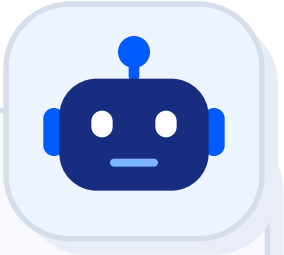
kubectl позволяет агенту управлять кластером



- Запускать приложения.
- Менять конфигурацию и число экземпляров.
- Обновлять версии и проверять результат.

Что произошло?
Агент помогает разобраться

После обновления калькулятор перестал отвечать. Почему?



Состояние объектов

События (Events)

Логи

Наблюдения → вероятная причина → проверка

Спасибо за внимание

01 Зачем приложению Kubernetes ✓

02 Запускаем приложение — Pod ✓

03 Даём постоянный адрес — Service ✓

04 Поддерживаем число экземпляров — Deployment ✓

05 Обновляем приложение ✓

06 ИИ-агент в K8S ✓